HashiCorp Certified: Terraform Associate（004）　目標 6d　#13

ドリフトとステートの整理

ドリフト（構成とステート）、ノーマルモードと refresh-only、非推奨の terraform refresh、HCP Terraform のドリフト検出、moved ブロック（名前・モジュール・インスタンスキー・つなげ方）、removed ブロックと state rm、ステートの分割と「削除してインポート」

各ページの下に、根拠にした資料（HashiCorp の Exam Content List（004）、Terraform ドキュメント v1.12.x の Command: plan・Command: refresh・moved block reference・Refactor modules・removed block reference・Remove a resource from state・Refactor Terraform state、HCP Terraform の Health assessments）を書いた。英語の原文は『』で引用した

第1章

ドリフト

外で変わったら、もどすか、受け入れるか。

ドリフトへの 2 つの対応

外での変更（コンソールなど）
実物 ≠ ステート・設定

↓ terraform plan（最初に refresh）

いらない変更 → 上書き
そのまま plan・apply で設定にもどす

残したい変更 → 設定を書きかえる
plan で差分なしを確かめる

> ステートだけ古い（設定と食いちがわない）なら apply -refresh-only。

種類	中身	直し方
構成のドリフト	外での変更が設定と食いちがう	上書き or 設定を変更
ステートのドリフト	設定とは食いちがわない	refresh-only

HCP Terraform のドリフト検出
ヘルス評価で定期的に。Health > Drift。見るのは構成のドリフトだけ。設定を変えるのはワークスペースの管理者

出典：Health assessments（Drift detection）／Command: plan

refresh-only と terraform refresh

コマンド	実物	ステート	覚えどころ
terraform plan	—	—	refresh して、設定にもどす変更を計画
terraform apply	設定に合わせる	更新	ノーマルモード
terraform plan -refresh-only	変えない	変えない	外での変更を見るだけ
terraform apply -refresh-only	変えない	実物に合わせる	確認してから書く（すすめ）
terraform refresh	変えない	確認なしで書く	非推奨。= apply -refresh-only -auto-approve

terraform refresh の危険：資格情報がまちがっていると「全部消えた」と思いこみ、確認なしにステートから全部消すおそれ。

-refresh=false は実物の読み直しをとばす（外での変更を見のがす）。refresh-only とは同時に使えない。

出典：Command: refresh／Command: plan（Planning modes）

第2章

moved ブロック

名前を変えても、作り直さない。

moved の書き方

```
# 名前を変える
moved {
  from = aws_instance.a
  to   = aws_instance.b
}
# 子モジュールの中へ
moved {
  from = aws_instance.web
  to   = module.app.aws_instance.web
}
# for_each を付ける
moved {
  from = aws_instance.a
  to   = aws_instance.a["small"]
}
# count から for_each へ
moved {
  from = aws_instance.a[0]
  to   = aws_instance.a["small"]
}
```

moved なしでラベルを変えると、古いものを消して新しいものを作る計画になる。

1. 計画の前に from がステートにあるか確かめる

↓

2. あれば to に名前を変える

↓

3. 最初から to で作ったかのように計画

count を新しく付けたときだけは、moved がなくても [0] に自動で移す（でも moved を書くのがすすめ）。

resource を data に変えることはできない。v1.1 から。

出典：moved block reference／Refactor modules

moved の残し方と state mv

```
# 2 回動かしたら、つなげて残す
moved {
  from = aws_instance.a
  to   = aws_instance.b
}
moved {
  from = aws_instance.b
  to   = aws_instance.c
}
```

『Removing a moved block is a breaking change』：古いアドレスの利用者は「消す」計画になる。共有モジュールでは過去の moved を全部残す。

	moved ブロック	terraform state mv
書く場所	設定	CLI
確かめ方	plan で見られる	すぐ書きかわる
記録	設定に残る・利用者に伝わる	残らない
位置づけ	すすめ（v1.1〜）	古いやり方

どちらもステートの中のアドレスを変えるだけで、実物の名前やタグは変えない。

出典：Refactor modules

第3章

外す・分ける

removed と import で、履歴を残して移す。

removed ブロック

```
# resource ブロックをこれに置きかえて apply
removed {
  from = aws_instance.example   # インスタンスキーは不可
  lifecycle {
    destroy = false             # 実物は残す
  }
}
```

やりたいこと	方法
管理をやめ、実物は残す	removed + destroy = false
実物も消す	resource を消して apply（removed の既定も消す）
もう一度管理	import（#14）

lifecycle で使えるのは destroy だけ。v1.7 から。

外す前に、そのリソースへの参照を消す（terraform validate で探せる）。

state rm より removed：『the removed block lets you preview the results of the operation』

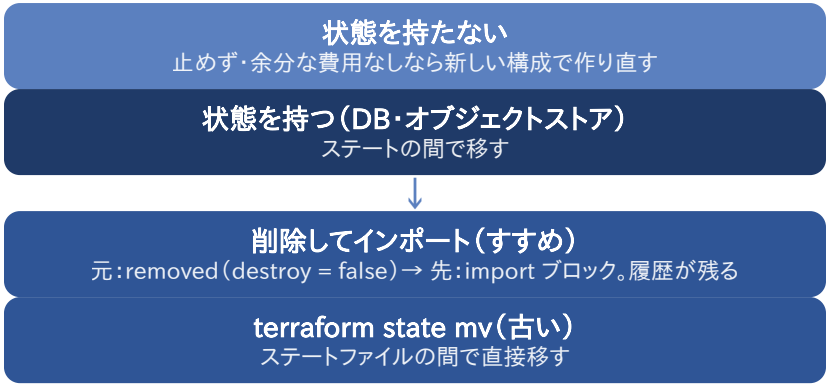
管理の引きわたし
最初の構築だけ Terraform、あとはほかのツール・チームへ

出典：removed block reference／Remove a resource from state

ステートの分割

分ける目安	例
変化の速さ	台数は毎日、ネットワークは何か月も同じ
状態を持つか	DB とサーバーを分ける（作り直しの事故を防ぐ）
担当のチーム	チームごとにワークスペース

参照は動的に：データソース・tfe_outputs・terraform_remote_state（IDを直接書かない）。依存は terraform graph で見る。



出典：Refactor Terraform state

本番での問われ方

ねらい	見るところ
ドリフト	plan が見つけて設定にもどす、残すなら設定を変える、HCP のドリフト検出
refresh-only	ステートと出力だけ更新、plan は見るだけ、refresh は非推奨で確認なし
moved	from・to、作り直しを防ぐ、インスタンスキー、消すと破壊的、つなげる、state mv との比較
removed	destroy = false、既定は消す、キー不可、state rm より安全、戻すなら import
分割	分ける目安、動的な参照、削除してインポート

出典：Exam Content List（004）6d／本資料の問題演習（全 30 問）